



EC-200 Data Structures

LAB MANUAL # 08

Course Instructor: Lecturer Anum Abdul Salam

Lab Engineer: Lab Engineer Ayesha Batool

Student Name: _____

Degree/ Syndicate: _____

	Trait	Obtained Marks	Maximum Marks
R1	Application Functionality 20%		20
R2	Specification & Data structure implementation 30%		30
R3	Reusability 10%		10
R4	Input Validation 10%		10
R5	Efficiency 20%		20
R6	Delivery 10%		10
R7	Plagiarism above 80%		1
Total			10

$$\text{Total Marks} = \text{Obtained Marks } (\sum_1^6 R_i * R_7)$$

LAB # 08: RECURSION

Lab Objective:

To Practice recursion/ think recursively.

Lab Description:

A Recursive task is a task that is defined in the terms of itself. Recursive function calls itself. With each invocation, the problem is reduced to a smaller task (**reducing case**) until the task arrives at some **terminal case**. A recursive function has two parts:

1. Terminal/base case. a stopping conditions.
2. Reducing case/recursive step an expression of the computation or definition in terms of itself.

Example

```
if (terminal_condition)
    terminal_case
else
    reducing_case
```

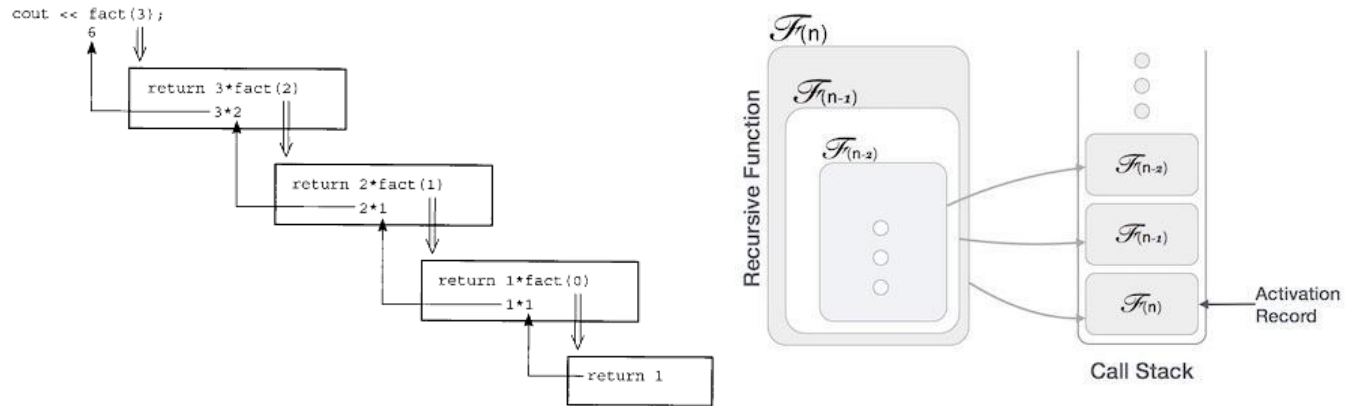
The factorial function:

$n! = n * (n-1) * (n-2) * \dots * 2 * 1$. The same function can be defined recursively as follows:

1. **$0! = 1$** – terminal case
2. $n! = n * (n - 1)!$ - the reducing case

Example

```
int fact (int n)
if (n==0) // base case
    return 1;
else      // reducing case
    return n*fact(n-1)
```



Activation Records:

Programming languages typically have local variables. That is created upon entry to function and Destroyed when function returns. Each invocation of a function has its own instantiation of local variables. Recursive calls to a function require several instantiations to exist simultaneously. Functions return only after all functions it calls have returned last-in-first-out (LIFO) behavior. A LIFO structure called a stack is used to hold each instantiation. The portion of the stack used for an invocation of a function is called the function's stack frame or activation record.

Example:

Let us consider the above factorial example to understand activation records.

step	Function call	Action	Stack state
1.	Fact(3)	Call generated from main	
2.	n*Fact(2)	Call generated from fact(3). Fact(3) pushed on stack.	Stack Frame for fact n=3
3	n*Fact(1)	Call generated from Fact (2). Fact (2) pushed on stack.	Stack Frame for fact n=3 Stack Frame for fact n=2
4	n*Fact(0)	Call generated from Fact (1). Fact (1) pushed on stack.	
5	1*1	Return 1 from base case. Fact(1) is popped from stack	Stack Frame for fact n=3 Stack Frame n=2

6	2*1	Fact(2) popped from stack	Stack Frame for fact n=3
7	3*2	Fact(3) popped from stack	Stack empty

Recursion elimination using stacks:

Recursion is implemented in via the system stack. Each recursive call to a method requires that the following information be pushed onto the system stack:

- Formal Parameters
- Local Variables
- Return Address

This information, collectively, is referred to as a stack frame. The stack frame for each method call will be different. For example, the stack frame for a parameter less/no local variable method will contain just a return address. All this is transparent to us as users because this happens magically due to the compiler. When we make a call to a recursive method, the compiler has taken care of the entire above required stack maintenance. To aid in our understanding of the process that takes place during the execution of a typical recursive routine, it is instructional to simulate it by managing this stack frame ourselves (not let C++/Java do it) with our own Stack. The following strategy can be used to the remove the recursion from a recursive routine, although not elegantly. There might be a far more pleasing method for a particular routine, but this technique is very instructional. It allows you simulate the system stack by declaring your own stack structure and manage the recursion. It is accomplished as follows:

1. Each time a recursive call is made in the algorithm, push the necessary information onto your stack.
2. When you complete processing at this deeper level, pop the simulated stack frame and continue processing in the higher level at the point dictated by the return address popped from the stack.
3. Use an iterative control structure that loops until there are no return addresses left to pop from the stack.

The use of a switch statement might help with the return address location.

LAB TASKS

1. Write a recursive function to find the elements multiple of 3 and at index multiple of 2 from given array
2. Write a recursive function power (int n, int x) to find n^x .

3. Write a recursive function in linked list ADT search (node*ptr, int data) to search data in list.
4. Write a recursive function to print a stack in reverse order.
5. Write a function to Print alternate nodes of a linked list using recursion
6. Write a function to display a singly list in reverse order and eliminate recursion using stack ADT you implemented in Lab # 06.
7. Your linked list contains following nodes: [5 4 3 2 1 0]

What is output of the following code? Write the complete steps for dry run and also show how each copy is maintained after each recursive call and tell what is the restriction in given function?

```
void fun (node* head) {
    if (head == NULL)
        return;
    head = head->next;
    cout << head->data << endl;
    if (head->next != NULL)
        fun(head->next);
    cout << head->data << endl;
}
```

8. Write a recursive function to sort a queue in descending order using Queue ADT you implemented in Lab # 07.

TEST PLAN:

1. **Execute your test plan. If you discover mistakes in your implementation, correct them and execute your test plan again.**

Sr.	Operations	Expected Results	Results/Status
1.	TASK01: Input: [11 36 69 43 15 34 67 72 68]	Output: 11 36 69 15 67 72 68	
2.	Create an empty list myList	head=NULL	
3.	Insert numbers 1-5 in list		
4.	Traverse myList	1,2,3,4,5	
5.	Search 6 in the list	Element not found	
6.	Search 2 in the list	Element found	

7.	Call power function power (2,0)	2	
8.	Call power function power (2,-1)	Power should be positive	
9.	Call power function power (2,2)	4	
10.	Traverse myStack backward	5,4,3,2,1	
11.	Delete from start of myList		
12.	Delete from start of myList		
13.	Delete from start of myList		
14.	Delete from start of myList		
15.	Delete from start of myList		
16.	Traverse myList forward	List empty	
17.	Traverse myList backward	List empty	
18.	Input LinkedList in task 4	1 2 3 4 5 6 7	
19.	alternativeNodes(Node* node, bool iseven=False)	1 3 5 7	
20.	alternativeNodes(Node* node, bool iseven=True)	2 4 6	
21.	Insert 5 elements in myQueue	5 8 3 9 7 2	
22.	sortMyQueue()	9 7 5 3 2	

THINK?

1. How can task 4 be done without using recursion? Implement without recursion
2. What if our recursive function has no base case?
3. What if recursive call is not approaching base case?
4. Which is better recursion or iteration? Why?